



Secure Secrets with Secrets Manager

CH

Arpita Chowdhury

```
Arpita Chowdhury and Arpita Chowdhury Remove hardcoded AWS credentials 50992e1 · 2 minutes ago History

Code Blame 20 lines (20 loc) · 883 Bytes

1 import boto3
2 import json
3 from botocore.exceptions import ClientError
4
5
6 def get_secret():
7     secret_name = "aws-access-key"
8     region_name = "us-east-1"
9
10    session = boto3.session.Session()
11    client = session.client(service_name="secretsmanager", region_name=region_name)
12
13    try:
14        response = client.get_secret_value(SecretId=secret_name)
15    except ClientError as e:
16        raise e
17
18    secret = response["SecretString"]
19    return json.loads(secret)
20
21
22 # Retrieve credentials from Secrets Manager
23 credentials = get_secret()
24
25 # Extract values (fallback region if missing)
26 AWS_ACCESS_KEY_ID = credentials.get("AWS_ACCESS_KEY_ID")
27 AWS_SECRET_ACCESS_KEY = credentials.get("AWS_SECRET_ACCESS_KEY")
28 AWS_REGION = credentials.get("AWS_REGION", boto3.session.Session().region_name or "us-east-1")

Symbols
Find definitions and references for functions and other symbols in this file by clicking a symbol below or in the code.
Filter symbols
func get_secret
const credentials
const AWS_ACCESS_KEY_ID
const AWS_SECRET_ACCESS_...
const AWS_REGION
```



Introducing Today's Project!

In this project, I will demonstrate cloning of a simple webapp that list S3 buckets, publicly share the app on GitHub, secure credential using AWS Secret Manager, update app code to use credential from AWS Secret Manager and see how I can wipe my repo clean of all the hardcoded credentials. I'm doing this project to learn to learn how to secure secrets with AWS Secret Manager.

Tools and concepts

Services I used were AWS Secrets Manager, AWS IAM, and GitHub, along with Python libraries like Boto3 and FastAPI. Key concepts I learnt include secure secrets management, avoiding hard-coded credentials, using IAM roles and policies for least-privilege access, GitHub secret scanning and push protection, and how to safely rewrite Git history using rebasing to remove sensitive data.

Project reflection

This project took me approximately 3 hours. The most challenging part was pushing the changes. It was most rewarding to finally see the changes in the git repo.



Arpita Chowdhury

NextWork Student

nextwork.org

I chose to do this project today because I wanted hands-on experience with real-world cloud security practices, especially learning how to manage secrets securely and fix common mistakes like accidentally committing credentials. Something that would make learning with NextWork even better is having short checkpoints or visual summaries after each step to reinforce key concepts and help learners quickly confirm they're on the right track.



Hardcoding credentials

In this project, a sample web app is exposing `AWS_ACCESS_KEY_ID` and `AWS_SECRET_ACCESS_KEY`. It is unsafe to hardcode credentials because it is a major security risk and is extremely unsafe for production applications. Once someone else gets access to your credentials, they can use them to access to AWS account, delete resources, steal data, and cause damage.

I've set up the initial configuration with hard-coded AWS credentials so the application can run during early testing. These credentials are just examples because hard-coding secrets is insecure and they will be replaced with a secure secrets management solution before production.



```
nextwork-security-secretsmanager — nano config.py — 80x24
UW PICO 5.09                                     File: config.py

# config.py - TEMPORARY for demonstration only
# WARNING: This is NOT safe for production! We'll fix it with Secrets Manager.

AWS_ACCESS_KEY_ID = "AKIAW3MEFRAFTQM5FHKE"
AWS_SECRET_ACCESS_KEY = "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"
AWS_REGION = "us-east-2"

[ Wrote 7 lines ]

^G Get Help  ^O WriteOut  ^R Read File  ^Y Prev Pg   ^K Cut Text   ^C Cur Pos
^X Exit      ^J Justify   ^W Where is   ^V Next Pg   ^U UnCut Text ^T To Spell
```



Using my own AWS credentials

As an extension for this project, I also decided to set up a Python virtual environment to ensure dependency isolation. Within the environment, I installed all required packages, including FastAPI, Uvicorn, and Boto3, to support the application and AWS service interactions.

When I first ran the app, I ran into an error because the application was using placeholder AWS credentials from config.py, which are not valid. As a result, AWS could not authenticate the request and returned an `InvalidAccessKeyId` error when the app attempted to access AWS services.

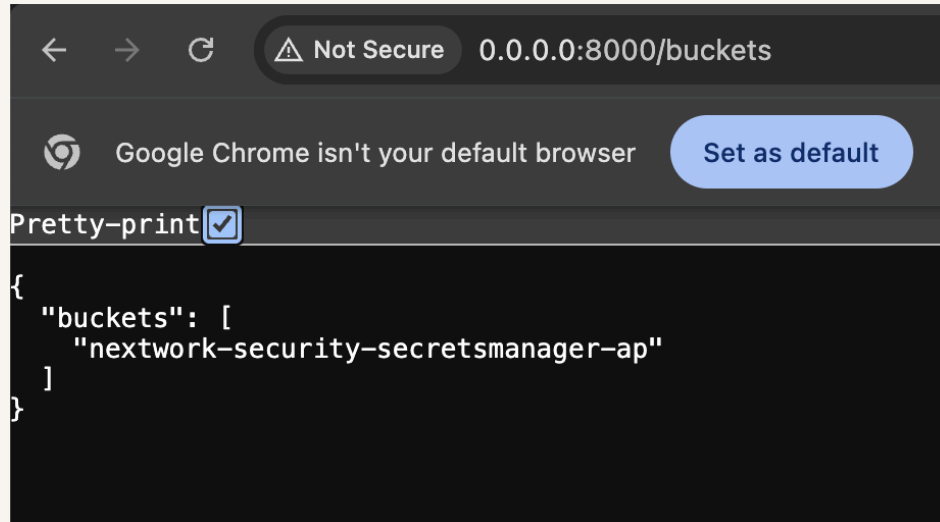
To resolve the 'InvalidAccessKeyId' error, I updated by creating a access key in my already existng IAM user where I replaced all the info with the I had in the .csv I downloaded after creating a access key.



Arpita Chowdhury

NextWork Student

nextwork.org





Pushing Insecure Code to GitHub

Once I updated the web app code with credentials, I forked the repository because it allowed me to create my own online copy of the project under my GitHub account and safely experiment without affecting the original repository. A fork is different from a clone because a fork creates a public copy of the repository on GitHub that lives in your account, while a clone creates a private, local copy of the repository on your computer for development and testing.

To connect my local repository to the forked repository, I added the fork's GitHub URL as the remote origin using Git. Then I used `git add` and `git commit` to stage and save my local changes with a descriptive commit message. Finally, `git push` uploads the committed changes from my local repository to my forked repository on GitHub.

GitHub blocked my push because Push Protection (secret scanning) detected AWS credentials in my commit specifically an AWS Access Key ID and AWS Secret Access Key inside `config.py` (lines 4–5) and rejected the push due to repository rule violations. This is a good security feature because it prevents sensitive secrets from being uploaded to GitHub (even by accident), which reduces the risk of someone stealing the keys, accessing AWS resources, and causing data breaches or unexpected charges.



```
nextwork-security-secretsmanager — git-remote-https • git push -u origin main — 109x31
create mode 100644 venv/lib/python3.9/site-packages/uvicorn/protocols/http/auto.py
create mode 100644 venv/lib/python3.9/site-packages/uvicorn/protocols/http/flow_control.py
create mode 100644 venv/lib/python3.9/site-packages/uvicorn/protocols/http/h11_impl.py
create mode 100644 venv/lib/python3.9/site-packages/uvicorn/protocols/http/httptools_impl.py
create mode 100644 venv/lib/python3.9/site-packages/uvicorn/protocols/utlis.py
create mode 100644 venv/lib/python3.9/site-packages/uvicorn/protocols/websockets/_init_.py
create mode 100644 venv/lib/python3.9/site-packages/uvicorn/protocols/websockets/auto.py
create mode 100644 venv/lib/python3.9/site-packages/uvicorn/protocols/websockets/websockets_impl.py
create mode 100644 venv/lib/python3.9/site-packages/uvicorn/protocols/websockets/wsproto_impl.py
create mode 100644 venv/lib/python3.9/site-packages/uvicorn/py.typed
create mode 100644 venv/lib/python3.9/site-packages/uvicorn/server.py
create mode 100644 venv/lib/python3.9/site-packages/uvicorn/supervisors/_init_.py
create mode 100644 venv/lib/python3.9/site-packages/uvicorn/supervisors/basereload.py
create mode 100644 venv/lib/python3.9/site-packages/uvicorn/supervisors/multiprocess.py
create mode 100644 venv/lib/python3.9/site-packages/uvicorn/supervisors/statreload.py
create mode 100644 venv/lib/python3.9/site-packages/uvicorn/supervisors/watchfilesreload.py
create mode 100644 venv/lib/python3.9/site-packages/uvicorn/workers.py
create mode 100644 venv/pyvenv.cfg
(venv) arpitachowdhury@Arpitas-MacBook-Air nextwork-security-secretsmanager % git push -u origin main
Username for 'https://github.com': Chyarpsh
Password for 'https://Chyarpsh@github.com':
remote: Invalid username or token. Password authentication is not supported for Git operations.
fatal: Authentication failed for 'https://github.com/Chyarpsh/nextwork-security-secretsmanager.git/'
(venv) arpitachowdhury@Arpitas-MacBook-Air nextwork-security-secretsmanager % git push -u origin main
Username for 'https://github.com': Chyarpsh
Password for 'https://Chyarpsh@github.com':
remote: Invalid username or token. Password authentication is not supported for Git operations.
fatal: Authentication failed for 'https://github.com/Chyarpsh/nextwork-security-secretsmanager.git/'
(venv) arpitachowdhury@Arpitas-MacBook-Air nextwork-security-secretsmanager % git push -u origin main
Username for 'https://github.com': Chyarpsh
Password for 'https://Chyarpsh@github.com': ?
```



Secrets Manager

Secrets Manager is an AWS service that securely stores, manages, and retrieves sensitive information such as credentials, API keys, and passwords. I'm using it to store AWS access credentials for my application so they are not hard-coded in the source code and can be accessed securely at runtime. Other common use cases include storing database usernames and passwords, API tokens for third-party services, encryption keys, and automatically rotating secrets without changing application code.

Another feature in Secrets Manager is automatic secret rotation, which means secrets such as passwords or access keys can be changed on a schedule without requiring manual updates to the application. It's useful in situations where long-lived credentials would otherwise increase security risk, such as database passwords or API keys, because regular rotation limits the impact of compromised secrets and helps meet security best practices and compliance requirements.

Secrets Manager provides sample code in various languages, like Python, Java, JavaScript (Node.js), and Go. This is helpful because it makes it easier for developers to integrate Secrets Manager into their applications quickly and correctly, reducing setup time and the risk of misconfiguring secure secret access.



```
1 // Use this code snippet in your app.
2 // If you need more information about configurations or implementing the sample
3 // code, visit the AWS docs:
4 // https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/home.html
5
6 // Make sure to import the following packages in your code
7 // import software.amazon.awssdk.regions.Region;
8 // import software.amazon.awssdk.services.secretsmanager.SecretsManagerClient;
9 // import software.amazon.awssdk.services.secretsmanager.model.GetSecretValueRequest;
10 // import software.amazon.awssdk.services.secretsmanager.model.GetSecretValueResponse;
11
12 public static void getSecret() {
13
14     String secretName = "aws-access-key";
15     Region region = Region.of("us-east-1");
```

Java Line 1, Column 1  Errors: 0 |  Warnings: 0



Updating the web app code

I updated the `config.py` file to retrieve AWS credentials securely from AWS Secrets Manager instead of hard-coding them in the source code. The `get_secret()` function will connect to AWS Secrets Manager using Boto3, request the secret named `aws-access-key` from the specified region, and return the stored secret value so the application can access credentials securely at runtime.

I also added code to `config.py` to extract the AWS Access Key ID, AWS Secret Access Key, and AWS region from the JSON returned by Secrets Manager. This is important because it allows the app to use the correct credential values at runtime without hard-coding secrets in the codebase, which prevents accidental leaks (like pushing keys to GitHub) and follows security best practices.



```
nextwork-security-secretsmanager — nano config.py — 100x35
(venv) arpitachowdhury@Arpitas-MacBook-Air nextwork-security-secretsmanager % nano config.py
UW PICO 5.09 File: config.py Modified

import boto3
from botocore.exceptions import ClientError

def get_secret():

    secret_name = "aws-access-key"
    region_name = "us-east-1"

    # Create a Secrets Manager client
    session = boto3.session.Session()
    client = session.client(
        service_name='secretsmanager',
        region_name=region_name
    )

    try:
        get_secret_value_response = client.get_secret_value(
            SecretId=secret_name
        )
    except ClientError as e:
        # For a list of exceptions thrown, see
        # https://docs.aws.amazon.com/secretsmanager/latest/apireference/API_GetSecretValue.html
        raise e

    secret = get_secret_value_response['SecretString']
```



Rebasing the repository

Git rebasing is a way to rewrite a repository's commit history by editing, reordering, or removing commits. I used it to remove a commit that contained hard-coded AWS credentials from the project's history. This was necessary because even though the credentials were later removed from the code, GitHub's secret scanning still detected them in the commit history and blocked the push. Rebasing allowed me to permanently eliminate the sensitive data and safely push the repository.

A merge conflict occurred during rebasing because Git couldn't automatically combine changes from different commits that modified the same section of a file (in this case, `config.py`). I resolved the merge conflict by manually reviewing the conflicting code, keeping the correct and secure version (the one that retrieves credentials from AWS Secrets Manager), removing the outdated hard-coded credentials, saving the file, and then continuing the rebase.

Once the merge conflict was resolved, I verified that the application code was using the secure Secrets Manager-based configuration, that `config.py` no longer contained any hard-coded AWS credentials, and that the Git rebase completed successfully with a clean working tree before pushing the changes to GitHub.



```
To https://github.com/Chyarpsh/nextwork-security-secretsmanager.git
! [remote rejected] main -> main (push declined due to repository rule violations)
error: failed to push some refs to 'https://github.com/Chyarpsh/nextwork-security-secretsmanager.git'
(venv) arpita@chowdhury@Arpita-MacBook-Air nextwork-security-secretsmanager % git rebase
Current branch main is up to date.
(venv) arpita@chowdhury@Arpita-MacBook-Air nextwork-security-secretsmanager % git rebase -i --root

hint: Waiting for your editor to close the file...
pick 6c3a9ca # first commit
pick 7e6497c # Delete .DS_Store
pick 8c63f73 # Update config.py
pick 201e292 # Update config.py
pick 9186d38 # Update config.py
d 5ff55c7 # Updated config.py with hardcoded credentials
pick 4da2192 # Updated config.py with Secrets Manager credentials

# Rebase 4da2192 onto 22015ca (7 commands)
#
# Commands:
# p, pick <commit> = use commit
# r, reword <commit> = use commit, but edit the commit message
# e, edit <commit> = use commit, but stop for amending
# s, squash <commit> = use commit, but meld into previous commit
# f, fixup [-C | -c] <commit> = like "squash" but keep only the previous
#                               commit's log message, unless -C is used, in which case
#                               keep only this commit's message; -c is same as -C but
#                               opens the editor
# x, exec <command> = run command (the rest of the line) using shell
# b, break = stop here (continue rebase later with 'git rebase --continue')
# d, drop <commit> = remove commit
# l, label <label> = label current HEAD with a name
# t, reset <label> = reset HEAD to a label
# m, merge [-C <commit> | -c <commit>] <label> [# <oneline>]
#     create a merge commit using the original merge commit's
#     message (or the oneline, if no original merge commit was
#     specified); use -c <commit> to reword the commit message
# u, update-ref <ref> = track a placeholder for the <ref> to be updated
#                       to this position in the new commits. The <ref> is
#                       updated at the end of the rebase
#
# These lines can be re-ordered; they are executed from top to bottom.
#
# If you remove a line here THAT COMMIT WILL BE LOST.
#
# However, if you remove everything, the rebase will be aborted.
#
~
```



nextwork.org

The place to learn & showcase your skills

Check out nextwork.org for more projects

